INTRODUCTION TO ROBOTICS FOR AI & DATA SCIENCE

Lab 1: Hello, Robot! Your First Step to Autonomy

Implementation Guide & Code Walkthrough

Dr. Akram Fatayer

Al-Zaytoonah University of Jordan

a.fatayer@zuj.edu.jo

linkedin.com/in/akram-fatayer/

This Lab Builds the Foundation of Your Autonomous System

The Primitives of Autonomy

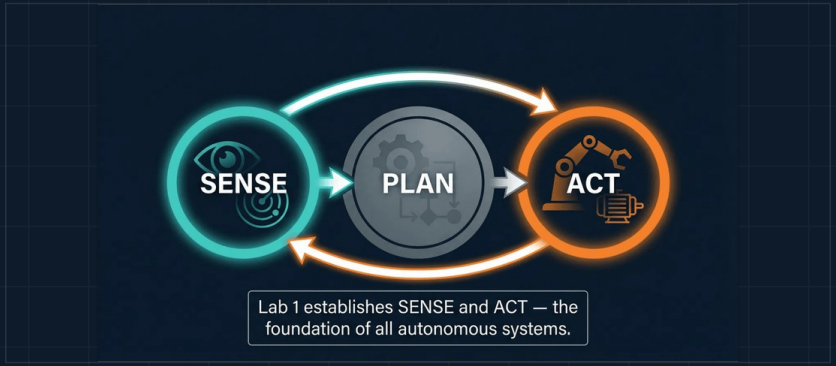
By the end of today, you will have a robot that can: spawn in a simulation, read its own state, and move in a controlled pattern.

Sense & Act

This lab establishes the **"Sense"** and **"Act"** components of the Sense-Plan-Act cycle. Everything we do from now on builds upon this foundation.

Real-World Relevance

These are the exact same primitive operations used in real autonomous vehicles, drones, and industrial robotic arms.



Lab 1 establishes SENSE and ACT — the foundation of all autonomous systems.

Each Student Creates a Unique, Isolated Conda Environment

Why Unique Environments?

Shared lab computers mean shared conflicts. Creating your own environment ensures your code runs exactly the same way every time, unaffected by other students' changes.

Why conda-forge?

Conda manages both Python and non-Python dependencies together. The **conda-forge** channel provides the most reliable, up-to-date scientific packages (like PyBullet).

The Golden Rule

Always try `conda install` first. Only use `pip install` as a fallback if a package is not available on Conda.

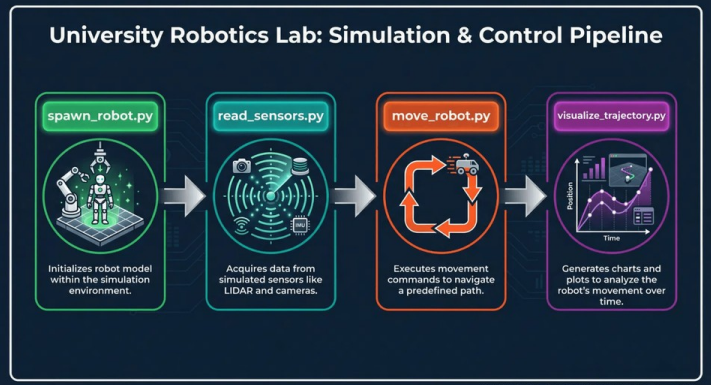
bash - conda setup

```
# 1. Create your unique environment (Do this ONCE)
$ conda create -n robotics_Ahmad python=3.11 -y

# 2. Activate it (Do this EVERY TIME you open a terminal)
$ conda activate robotics_Ahmad

# 3. Install required packages from conda-forge
(robotics_Ahmad) $ conda install -c conda-forge pybullet numpy matplotlib
opencv -y
```

Four Scripts, Four Skills, One Complete Foundation



Incremental Integration

Each script builds directly on the previous one. You start by creating the world, then sensing it, acting within it, and finally analyzing your performance. This is the core methodology of the course.

Automated Self-Grading

Plus: `check_my_lab.py`

Before submission, run this script to get instant feedback. It verifies your robot's trajectory, reinforcing test-driven development principles.

spawn_robot.py: Five Steps to Create a Simulated World

1 — Connect to Physics Engine

Use `p.GUI` for visual, or `p.DIRECT` for headless.

2 — Set Search Path

Tell PyBullet where to find `plane.urdf` and `r2d2.urdf`.

3 — Set Gravity

Establish gravity (Z = -9.8 m/s²).

4 — Load URDF Models

Load robot & plane URDFs to the world.

5 — Run Simulation Loop

Advance the physics one step at a time.

Expected Output: A GUI showing R2D2 on a plane.

spawn_robot.py

```
import pybullet as p
import pybullet_data

# 1. Connect
p.connect(p.GUI)

# 2. Search path
p.setAdditionalSearchPath(pybullet_data.getDataPath())

# 3. Gravity
p.setGravity(0,0,-9.8)

# 4. Load URDFs
p.loadURDF("plane.urdf")
p.loadURDF("r2d2.urdf", [0,0,0.1])

# 5. Run loop
for _ in range(1200):
    p.stepSimulation()
```

read_sensors.py : Teaching Your Robot to "Feel" Its Own State

Proprioceptive Sensing

Sensing the robot's own **internal state** (position, orientation, speed). This is how the robot knows where its body is in space.

Exteroceptive Sensing

Sensing the **external world** (cameras, lidar, ultrasonic). We will cover this in future labs when we add "eyes" to the robot.

The Golden Rule

You cannot control what you cannot measure. Before we can make the robot move accurately, we must be able to read its current state.

```
pos, orn = p.getBasePositionAndOrientation(robotId)
```

Returns the (x, y, z) position and the orientation as a quaternion (x, y, z, w).

```
lin_vel, ang_vel = p.getBaseVelocity(robotId)
```

Returns linear velocity (vx, vy, vz) and angular velocity (wx, wy, wz).

```
euler = p.getEulerFromQuaternion(orn)
```

Converts complex quaternions into human-readable (roll, pitch, yaw) angles.

Expected Console Output (Every 0.5s)

```
Time: 2.50s
Position (x,y,z): (0.000, 0.000, 0.500)
Orientation (r,p,y): (0.000, 0.000, 0.000)
Linear Velocity: (0.000, 0.000, 0.000)
Angular Velocity: (0.000, 0.000, 0.000)
```

move_robot.py: Commanding Motion with resetBaseVelocity

The Core Function

```
import pybullet as p
# Directly sets the robot body's velocity
p.resetBaseVelocity(robotId, linearVelocity=[vx, vy, 0], angularVelocity=[0, 0, wz])
```

This function directly commands the robot's base. It is simple, reliable, and perfect for learning the fundamentals of motion control before dealing with complex wheel physics.

The Body-to-World Frame Problem

- **Robot thinks:** "Move forward/backward" (Body Frame)
- **Physics engine thinks:** "Move along X/Y/Z" (World Frame)
- **Solution:** Use the current yaw angle to convert: vx = speed * cos(yaw), vy = speed * sin(yaw)

Movement Parameters (2x2m Square)

1.0 m/s LINEAR SPEED	**2.0 s** MOVE DURATION
π/2 rad/s TURN SPEED (90°/S)	**1.0 s** TURN DURATION

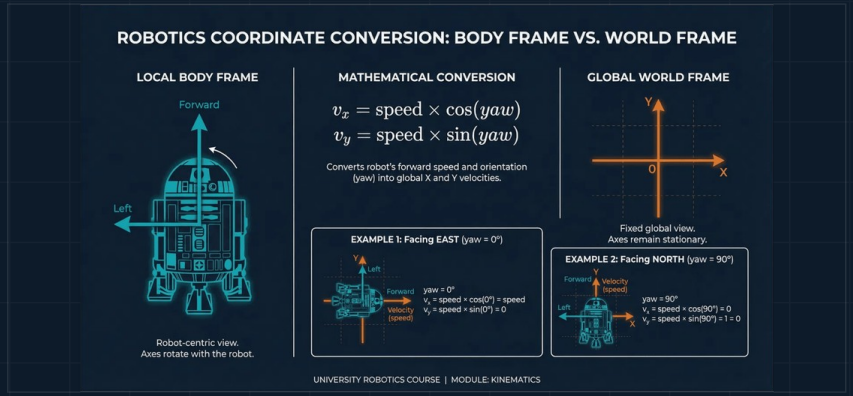
The Control Loop

To draw a square, we repeat the sequence 4 times:

For i in range(4) → **Move Forward (2s)** → **Turn Left (1s)**

Expected Output: The robot traces a visible square in the GUI window and returns to its starting position.

Why cos(yaw) and sin(yaw)? Converting Body Frame to World Frame



The Core Problem

The robot thinks in "forward/backward" (**Body Frame**), but the physics engine thinks in absolute "X/Y/Z" coordinates (**World Frame**).

The Trigonometric Solution

We use the robot's current heading (`yaw`) to convert its forward speed into global X and Y velocities:

Facing East (yaw=0):

vx = speed × cos(0) = speed

vy = speed × sin(0) = 0

Key Insight

The robot's "forward" direction changes as it turns, but the world's X and Y axes stay fixed. This is why we must recalculate vx and vy at *every simulation step*.

visualize_trajectory.py: From Raw Data to Actionable Insights

Step 1: Data Logging

Instead of just watching the robot move, we need to record its position at every step to analyze its performance later.

```
# Initialize empty list trajectory = [] # Inside simulation loop: pos, _ = p.getBasePositionAndOrientation(robotId) trajectory.append([pos[0], pos[1], pos[2]])
```

Headless Mode (p.DIRECT)

When collecting data, we don't need the GUI. Using p.DIRECT runs the simulation thousands of times faster in the background.

Step 2: Plotting with Matplotlib

We convert the list into a NumPy array and plot the X and Y coordinates to visualize the path.

```
import matplotlib.pyplot as plt import numpy as np traj_array = np.array(trajectory) plt.plot(traj_array[:, 0], traj_array[:, 1]) plt.show()
```

Generates a 2D Top-Down View of the Square Path

Verify Your Work Before Submission

Automated Testing

In industry, code is never deployed without automated tests. We introduce this concept in Lab 1 with the `check_my_lab.py` script.

What It Checks

- **Shape:** Did the robot make exactly 4 turns?
- **Distance:** Are the sides approximately 2 meters long?
- **Closure:** Did the robot return to its starting point?

Instant Feedback

Run this script as many times as you need. If all tests pass, you are guaranteed full marks for the coding portion of the lab.

Terminal

```
(robotics) user@zuj:~$ python check_my_lab.py

--- Running Automated Lab Check for Week 1 ---
Importing your 'move_square' function...
Running simulation in background...

[PASS] 4 Corners Detected
Found exactly 4 distinct turning points.

[PASS] Side Lengths are ~2m
Measured sides: ['2.01', '1.98', '2.03', '1.99']

[PASS] Returned to Origin
Final distance from start: 0.05m (Threshold: 0.2m)

========================================
FINAL RESULT: 3/3 TESTS PASSED
========================================
Ready for submission!
```

What to Submit and How to Succeed

Submission Checklist

- **move_robot.py**
  Your completed script that successfully drives the robot in a 2x2m square.
- **trajectory.png**
  The plot generated by `visualize_trajectory.py` showing your robot's path.
- **check_output.txt**
  A screenshot or text copy showing that your code passed all tests in `check_my_lab.py`.
- **Zip File Format**
  Package all files into a single zip named `Lab1_YourName_YourID.zip`.

Pro Tips for Success

- **Read the Comments:** The provided code is heavily commented. Read them to understand the *why*, not just the *how*.
- **Test Incrementally:** Don't write the whole square at once. Make it move forward first. Then make it turn. Then combine them.
- **Mind the Units:** PyBullet uses meters for distance and radians for angles. (90 degrees = π/2 radians ≈ 1.57).
- **Use Your Unique Environment:** Always remember to `conda activate robotics_YourName` before running any scripts.

You Just Built the Foundation of an Autonomous System

Today you learned to spawn a world, read sensors, command motion, and analyze data. Next week, we give the robot "eyes" with Computer Vision.

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo

linkedin.com/in/akram-fatayer